ProSEIT

ENGINEERING CONTROL SERIES

ProSEIT PAEP

Claude Engineering Execution & Quality Assurance Protocol

CONTROLLED AI-ASSISTED SOFTWARE ENGINEERING OPERATING MANUAL

DOCUMENT ID	PROSEIT-STD-PMRA-AF-006-EQP01	**VERSION**	1.1
ISSUE DATE	23 September 2026	**STATUS**	Review-ready controlled protocol - Design-to-Code amendment
OWNER	Solutions Architect & Main Product Owner	**DEVELOPING ORGANISATION**	PROSEIT ORGANISATION OF SOFTWARE PROFESSIONALS LTD
PARENT ARCHITECTURE	PAEP v1.1 - PROSEIT-STD-PMRA-AF-006	**IMPLEMENTATION BASELINE**	PAEP-IB-01 v1.0
PRIMARY EXECUTION AGENT	Claude / Claude Code under repository controls	**APPROVAL**	Solutions Architect + Main Product Owner

PURPOSE OF THIS COPY: Establishes how Claude shall plan, implement, verify, repair, evidence and hand over PAEP frontend work packages. A work package may be marked complete only after all mandatory acceptance gates pass and the required evidence is retained.

Document Control

Field	Controlled Value
Document title	ProSEIT PAEP Claude Engineering Execution & Quality Assurance Protocol
Document ID	PROSEIT-STD-PMRA-AF-006-EQP01 / PAEP-EQP-01
Version	1.1
Status	Review-ready controlled protocol - Design-to-Code amendment
Effective date	Upon Product Owner / Architecture approval
Parent architecture	PAEP v1.1 - PROSEIT-STD-PMRA-AF-006
Implementation baseline	PAEP-IB-01 - Implementation Architecture & Repository Baseline v1.0
Initial execution roadmap	PAEP-IR-01 - Implementation Roadmap IB0-IB2 v1.0
Primary execution agent	Claude / Claude Code operating under repository and CI controls
Backend decision	ERPNext + Frappe retained as the strategic backend; this protocol introduces no replacement
Main-text formatting	12 pt body text; each new enumerated list restarts at lowercase Roman (i)
Prompt formatting	Claude prompts are visually separated from normative body text
Code formatting	Code/configuration snippets are presented in monospace shaded blocks

Revision History

Version	Date	Disposition	Principal Change
1.0	15 Sep 2026	Review candidate	Initial controlled protocol for Claude-assisted PAEP engineering, QA, traceability, acceptance and fix-until-pass execution.
1.1	23 Sep 2026	Review candidate	Adds a controlled Design-to-Code Handoff stage; separates Claude Design design intent from Claude Code production implementation; introduces design-significant work classification, handoff approval, deviation control and the PAEP Design-to-Code Handoff Standard v1.0.

Approval and Decision Rights

Decision Domain	Primary Authority	Required Evidence / Review
PAEP architecture and exceptions	Solutions Architect	ADR, impact analysis, architecture review
Product priorities and acceptance	Main Product Owner	Work-package evidence, UAT/acceptance record
Automated technical PASS	CI quality gates	Reproducible command output and retained logs
Security/privacy release acceptance	Security and Assurance Authority / authorised reviewer	Threat model, scans, negative tests, exception register
Accessibility/UX acceptance	Frontend Platform Council / Quality Lead	Automated + manual evidence and representative-user checks
Claude implementation execution	Engineering team operating Claude	Traceability, implementation notes, tests and evidence

Decision Domain	Primary Authority	Required Evidence / Review
Release approval	Solutions Architect + Main Product Owner	Compatibility, migration, rollback, QA and provenance evidence

NORMATIVE RELATIONSHIP: This protocol is subordinate to PAEP v1.1 and PAEP-IB-01. It governs how AI-assisted implementation is executed and assured, but shall not weaken package boundaries, backend authority, security, portability, accessibility, clean-room provenance, reporting contracts or release controls established by the parent architecture.

Contents

PART I — CONTROL AND ENGINEERING POSITION	**PART V — QUALITY ASSURANCE DOMAINS**
Section 1 — Purpose, Scope and Status	Section 16 — Architecture and Modularity Assurance
Section 2 — Source-of-Truth Hierarchy	Section 17 — Security and Privacy Assurance
Section 3 — Normative Language and Principles	Section 18 — Accessibility, UX and Responsive Assurance
Section 4 — Roles and Decision Rights	Section 19 — Performance, Reliability and Observability
PART II — SOFTWARE ENGINEERING LIFECYCLE	Section 20 — Reporting and Analytics Assurance
Section 5 — Stage-Gated Engineering Lifecycle	Section 21 — Offline/PWA and Localisation Assurance
Section 6 — Engineering Stage Gates	Section 22 — Clean-Room, Dependencies and Provenance
Section 7 — Work Breakdown Structure	**PART VI — DEFECT, EVIDENCE AND CHANGE CONTROL**
PART III — CLAUDE EXECUTION OPERATING MODEL	Section 23 — Fix-Until-Pass Defect Protocol
Section 8 — Claude Role and Prohibited Behaviours	Section 24 — Evidence Repository and Retention
Section 9 — CLAUDE.md Repository Contract	Section 25 — Requirements Traceability
Section 10 — Prompt Engineering Standard	Section 26 — ADR and Architecture Exception Control
Section 11 — Work-Package Execution Cycle	Section 27 — Defect Classification and Escalation
PART IV — READINESS AND ACCEPTANCE	**PART VII — PAEP EXECUTION ROADMAP**
Section 12 — Definition of Ready	Section 28 — W0-W12 Controlled Work Breakdown
Section 13 — Definition of Done	Section 29 — IB0-IB2 Execution Protocol
Section 14 — Acceptance Criteria Standard	Section 30 — Post-IB2 Execution Controls
Section 15 — Quality Gate Matrix	**PART VIII — CONTROLLED TEMPLATES AND APPENDICES**
	Section 31 — Work-Package Control Template
	Section 32 — Acceptance Report Template
	Section 33 — Claude Prompt Library
	Section 34 — Release Readiness Decision

	Appendix A — Baseline CLAUDE.md
	Appendix B — QA Command Naming Standard
	Appendix C — Status Model
	Appendix D — Controlled References

PART I — CONTROL AND ENGINEERING POSITION

Section 1 — Purpose, Scope and Status

This Protocol defines the controlled operating method for using Claude or Claude Code to engineer the ProSEIT Angular Experience Platform (PAEP). It converts architectural intent into repeatable execution rules, testable acceptance criteria, evidence requirements and approval boundaries. Claude is treated as an implementation agent operating within ProSEIT engineering governance; it is not the architecture authority and it may not self-approve exceptions.

1.1 Purpose

(i) ensure every Claude-assisted work package starts from controlled requirements and ends with retained verification evidence;

(ii) prevent implementation convenience from eroding PAEP modularity, backend authority, portability, accessibility, security or clean-room provenance;

(iii) require objective, executable quality gates instead of conversational claims that code “looks complete”;

(iv) establish a fix-until-pass discipline for mandatory criteria;

(v) provide a standard prompt structure so Claude receives bounded engineering tasks rather than vague product-level requests; and

(vi) preserve human approval for architecture exceptions, product acceptance, usability judgement and controlled release decisions.

1.2 Scope

The Protocol applies to PAEP workspace creation, shared packages, design-system implementation, enterprise shell, product-neutral contracts, Frappe adapter, analytics/reporting, offline/PWA, internationalisation, domain kits, reference applications, testing utilities, CI/CD configuration, documentation and evidence. It also applies to later PAEP-derived product adoption where Claude generates or modifies shared-platform code.

1.3 Non-goals

(i) This Protocol does not authorise Claude to change PAEP architecture decisions without an approved ADR or architecture exception.

(ii) This Protocol does not make client-side tests a substitute for backend authorization, domain validation, fiscal controls or entitlement enforcement.

(iii) This Protocol does not permit direct external DocType CRUD, direct operational-database access from the browser, or vendor-specific analytics objects to become PAEP public contracts.

(iv) This Protocol does not replace human user-acceptance testing, representative-user usability assessment, legal/licence review or release authority.

Section 2 — Source-of-Truth Hierarchy

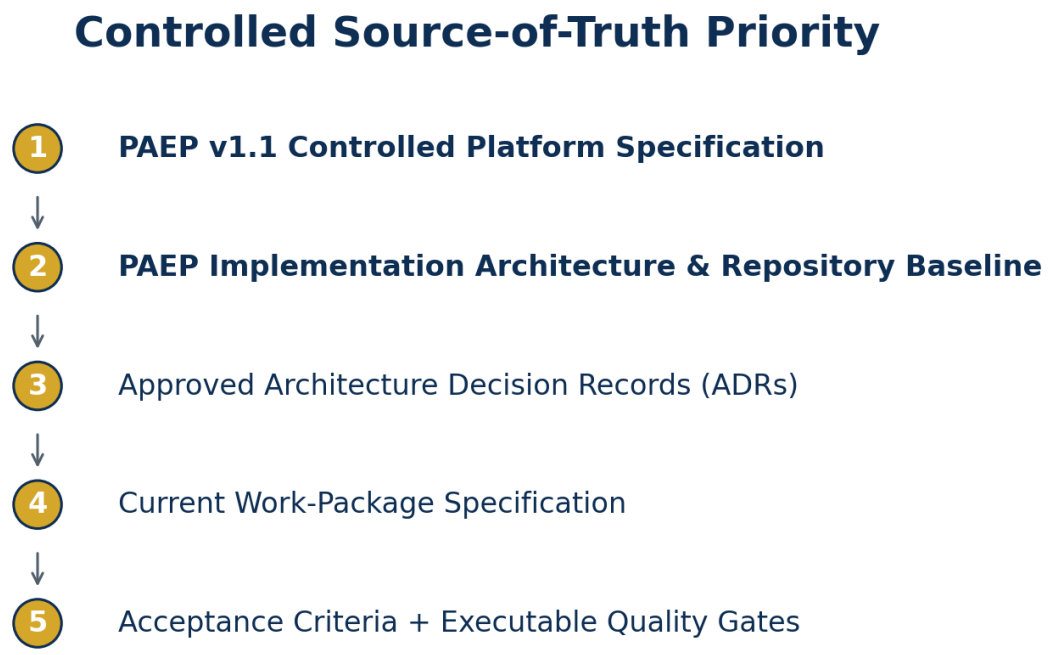


Figure 1. Controlled source-of-truth priority for Claude-assisted PAEP engineering.

When instructions appear inconsistent, Claude shall resolve them using the hierarchy above. A lower-level prompt may narrow a task but shall not relax higher-level controls. If a genuine conflict remains, Claude shall stop the affected work, identify the conflicting clauses and request an architecture decision rather than silently choosing one interpretation.

(i) PAEP v1.1 governs platform architecture, security boundaries, portability, accessibility, component architecture, reporting posture and release controls.

(ii) PAEP-IB-01 governs concrete repository boundaries, contract packages, runtime configuration and implementation structure.

(iii) Approved ADRs govern authorised deviations or decisions not fully resolved by the parent specifications.

(iv) The current work-package specification governs the bounded change to be implemented.

(v) Acceptance criteria and executable gates govern whether the implementation technically passes.

Section 3 — Normative Language and Principles

“Shall” and “must” denote mandatory controls. “Should” denotes a recommended control requiring documented justification where not applied. “May” denotes an optional implementation choice. A mandatory quality gate may be waived only through a recorded exception approved by the competent authority; Claude itself cannot waive a gate.

(i) architecture before screens;

(ii) contracts before backend coupling;

(iii) server authority before client convenience;

(iv) accessible and responsive behaviour by default;

(v) configuration before source forks;

(vi) portable public contracts before vendor-specific adapters;

(vii) quality evidence before completion claims;

(viii) fix the cause rather than suppress the gate; and

(ix) clean-room provenance and dependency obligations remain part of the engineering definition of done.

Section 4 — Roles and Decision Rights

Role	Primary Responsibility	May Approve
Solutions Architect	Architecture, package boundaries, portability, ADRs and exceptions	Architecture decisions and controlled exceptions
Main Product Owner	Product priorities, value, acceptance and release readiness	Product acceptance and release acceptance jointly as defined
Engineering Lead	Work-package planning, Claude execution supervision, integration and evidence completeness	Technical handover within delegated authority
Claude / Claude Code	Plan bounded tasks, generate/modify code, run gates, diagnose failures, repair and prepare evidence	Nothing that changes controlled architecture or waives gates
CI/CD Platform	Execute deterministic build, test, lint, security and quality gates	Technical PASS only when configured gate succeeds
Quality / Accessibility Reviewer	Manual and automated quality review	Assigned QA evidence
Security Reviewer	Threat-model contribution, security tests and deployment posture	Assigned security evidence
Representative User / UAT Participant	Workflow usability and product fitness	UAT evidence; not architecture exceptions

PART II — SOFTWARE ENGINEERING LIFECYCLE

Section 5 — Stage-Gated Engineering Lifecycle

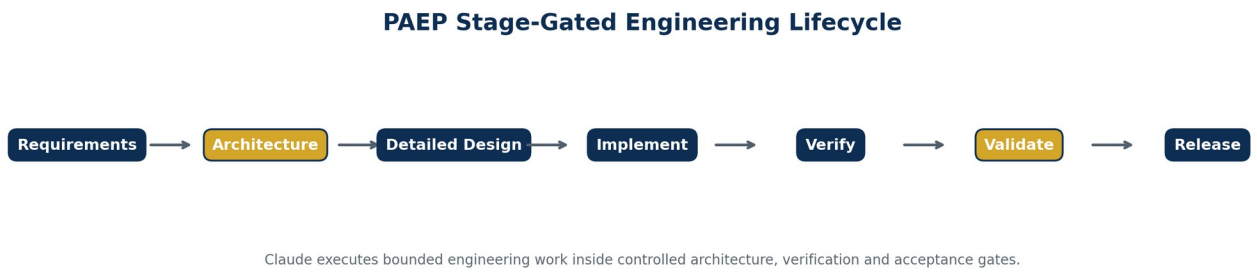


Figure 2. PAEP stage-gated software-engineering lifecycle.

Claude-generated code is only one activity within the PAEP lifecycle. The controlled product is the combination of requirements, architecture conformity, design rationale, implementation, tests, evidence, documentation, provenance and release controls. Work may iterate within a stage, but progression to the next stage requires the applicable exit evidence.

5.1 Lifecycle stages

(i) Requirements engineering — establish traceable functional, quality, security, accessibility, portability, reporting and deployment requirements.

(ii) Architecture — confirm package ownership, public contracts, trust boundaries, runtime configuration, backend authority and applicable ADRs.

(iii) Detailed design — define APIs, component states, interaction behaviour, responsive adaptations, test strategy and failure/recovery states.

(iv) Implementation — make the smallest bounded change that satisfies the approved design and work-package scope.

(v) Verification — run unit, component, contract, integration, accessibility, security, performance, architecture and regression gates as applicable.

(vi) Validation — confirm representative workflows, usability and business intent with appropriate human review/UAT.

(vii) Release — package only after compatibility, rollback, provenance, SBOM, migration and acceptance evidence are complete.

(viii) Maintenance and evolution — manage defects, deprecations, dependency updates, framework upgrades and new capabilities through the same control model.

5.2 Controlled Design-to-Code Handoff

(i) Design-significant PAEP work shall pass through a controlled design-to-code handoff before production implementation begins.

(ii) Claude Design may explore visual hierarchy, interaction flow, responsive variants and prototypes, but it shall not redefine PAEP package boundaries, security authority, backend contracts, tenant rules, reporting trust boundaries or release controls.

(iii) Claude Code shall implement only from an approved handoff package and the controlling PAEP specifications; prototype HTML or generated design code is reference material and shall not be treated as production architecture.

(iv) Minor component maintenance or implementation of an already-approved PAEP pattern may proceed directly in Claude Code when the work package records that no new design decision is introduced.

(v) Any material departure from an approved design shall be recorded in a Design Deviation Record and approved through the applicable design/architecture authority before the departure is treated as accepted.

(vi) The detailed handoff artifacts, status model, acceptance checklist and prompt controls are defined by the PAEP Design-to-Code Handoff Standard v1.0 (PAEP-DCH-01).

Section 6 — Engineering Stage Gates

Gate	Entry Condition	Exit Evidence
G0 Requirements Ready	Controlled source documents identified	Requirement IDs, scope, acceptance criteria, traceability
G1 Architecture Ready	Requirements accepted for implementation	Package ownership, dependency direction, ADR status, threat/quality considerations
G2 Design Ready	Architecture implications resolved	Interfaces, state model, interaction design, test plan, failure states
G3 Implementation Complete	Bounded code/config change implemented	Buildable change, implementation notes, no known intentional gate bypass
G4 Verification PASS	Mandatory automated/manual checks executed	All mandatory gates PASS; retained logs/results
G5 Validation Accepted	Verification PASS	UAT/usability evidence where required; Product Owner acceptance
G6 Release Ready	Acceptance and release evidence assembled	SBOM/provenance, compatibility, migration, rollback and release approval

Section 7 — Work Breakdown Structure

The PAEP implementation shall be executed through bounded work packages grouped under the W0-W12 structure. Claude should never receive an unbounded instruction such as “build the PAEP template.” Each prompt shall identify one milestone or a tightly related set of artifacts, its inputs, permitted files, acceptance criteria and mandatory quality commands.

WBS	Stage	Principal Outcome
W0	Governance, requirements and traceability	Controlled engineering baseline
W1 / IB0	Repository and toolchain	Buildable governed Angular workspace
W2 / IB1	Contracts and runtime configuration	Portable frontend contract layer
W3 / IB2	Foundation and core components	Design system + UI laboratory
W4	Enterprise shell	Reusable enterprise experience
W5	ERPNext/Frappe adapter	Contract-conforming backend integration
W6	Analytics and rich reporting	Power BI-class governed reporting experience
W7	PWA/offline/i18n/resilience	Field-ready operating profile
W8	Domain component kits	Reusable business interaction kits
W9	Golden vertical slice	End-to-end architecture proof
W10	System qualification	Security/performance/a11y/ compatibility proof
W11	Packaging and release	Controlled PAEP release
W12	Product adoption and evolution	Portfolio rollout and maintenance

PART III — CLAUDE EXECUTION OPERATING MODEL

Section 8 — Claude Role and Prohibited Behaviours

Claude acts as a supervised engineering executor. It may analyse controlled requirements, propose a bounded implementation plan, modify authorised files, generate tests, run tools, diagnose failures and prepare evidence. It shall not convert convenience into architecture policy.

8.1 Required behaviours

(i) read the current source-of-truth documents and the work-package brief before modifying code;

(ii) identify the requirement and acceptance IDs addressed by the change;

(iii) produce a short implementation plan before substantive modification;

(iv) prefer the smallest change that meets the requirement without unrelated refactoring;

(v) run every mandatory gate specified for the work package;

(vi) when a gate fails, diagnose and repair the cause, then rerun the failed gate and affected regression set;

(vii) produce a concise implementation and evidence summary; and

(viii) leave the work package IN PROGRESS if any mandatory criterion remains unresolved.

8.2 Prohibited behaviours

(i) changing controlled architecture decisions without an approved ADR or explicit architecture approval;

(ii) introducing direct UI-to-Frappe persistence coupling or unrestricted DocType access contrary to PAEP contracts;

(iii) introducing upward, circular, deep/internal or otherwise prohibited package dependencies;

(iv) embedding secrets, private keys, privileged credentials or analytical-store credentials in browser configuration;

(v) weakening, deleting, skipping or rewriting tests merely to obtain a passing result;

(vi) suppressing TypeScript, Angular, accessibility, security or lint errors without approved rationale;

(vii) representing client-side permission, feature or edition gating as authoritative security enforcement;

(viii) exposing SQL or vendor-specific reporting configuration as PAEP public reporting contracts;

(ix) copying proprietary commercial-template source, theme files, assets or distinctive implementation artefacts; and

(x) marking a work package COMPLETE while any mandatory quality gate is failing or unexecuted.

Section 9 — CLAUDE.md Repository Contract

The repository root shall contain a version-controlled CLAUDE.md defining the engineering rules Claude must obey. This file is an execution contract, not a substitute for PAEP v1.1. It should remain short enough to be reliably consumed while linking to controlled specifications and quality commands.

Baseline repository policy excerpt

```
# PAEP CLAUDE ENGINEERING CONTRACT

SOURCE-OF-TRUTH PRIORITY
1. PAEP v1.1 Controlled Platform Specification
2. PAEP Implementation Architecture & Repository Baseline
3. Approved ADRs
4. Current Work-Package Specification
5. Acceptance Criteria and Executable Quality Gates

MANDATORY RULE
A failed mandatory criterion means the work package remains IN PROGRESS.
Claude shall fix the cause and rerun the failed gate plus affected regression gates.
Only an approved exception may waive a mandatory gate.
```

CONTROL: Appendix A provides the complete recommended baseline CLAUDE.md. Product teams may add narrower instructions, but they shall not weaken higher-level PAEP controls.

Section 10 — Prompt Engineering Standard

Every implementation prompt shall be structured so Claude knows what is authoritative, what it may change, how success is measured and what evidence is required. Prompts are engineering work orders and should be versioned or retained alongside the work package where the change is material.

(i) Context — identify PAEP version, implementation baseline, milestone and relevant ADRs.

(ii) Objective — state one bounded engineering outcome.

(iii) Inputs — list source files, schemas, interfaces, designs or existing tests Claude must use.

(iv) Permitted change boundary — identify packages/directories Claude may modify.

(v) Architecture constraints — restate only the controls material to this work package.

(vi) Implementation tasks — enumerate the expected artifacts or behaviours.

(vii) Acceptance criteria — provide objective pass/fail conditions with IDs.

(viii) Mandatory quality commands — specify the commands Claude must run.

(ix) Evidence — define logs, screenshots, reports or summaries to retain.

(x) Stop conditions — identify unresolved conflicts, missing authority or unsupported behaviour that requires escalation.

10.1 Design Work Orders and Handoff Inputs

(i) A design work order shall identify screen/workflow IDs, user task, approved design-system baseline, viewport profiles, mandatory states, accessibility intent and known data assumptions.

(ii) Claude Design prompts shall request capability- and task-oriented design using the PAEP design system; prompts shall not request replication of proprietary screens or distinctive third-party template implementations.

(iii) The approved handoff package shall be listed as an implementation input when Claude Code receives a design-significant work package.

(iv) Claude Code shall report any ambiguity, missing state or conflict between the handoff and PAEP architecture rather than resolving it silently through new design or architecture decisions.

CLAUDE EXECUTION PROMPT — STANDARD WORK-PACKAGE PROMPT

```
You are implementing PAEP work package <WP-ID> under PAEP v1.1 and PAEP-IB-01.

OBJECTIVE
<one bounded outcome>

AUTHORITATIVE INPUTS
- <controlled specification sections>
- <approved ADRs>
- <existing interfaces/tests>

PERMITTED CHANGE BOUNDARY
- <packages/directories>

MANDATORY CONSTRAINTS
- Do not change controlled architecture decisions.
- Do not weaken, skip or delete quality gates to obtain PASS.
- Preserve package boundaries, backend authority, portability, accessibility and clean-room rules.

ACCEPTANCE CRITERIA
- <AC-01 ...>
```

- <AC-02 ...>

MANDATORY QUALITY GATES
- <command 1>
- <command 2>

EXECUTION RULE
Plan → implement the smallest bounded change → run all gates → if any fail, diagnose/fix/re-run → capture evidence.
Do not mark COMPLETE until every mandatory criterion is PASS. If an architecture conflict is discovered, stop and report it.

Section 11 — Work-Package Execution Cycle

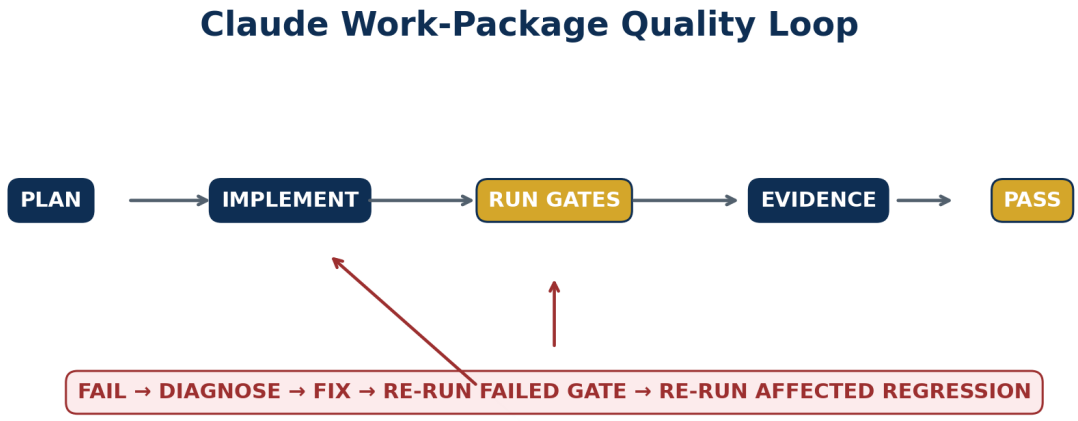


Figure 3. Mandatory Claude work-package fix-until-pass cycle.

(i) Read — load the work-package brief, relevant source-of-truth sections and existing tests.

(ii) Trace — map the change to requirement and acceptance IDs.

(iii) Plan — describe intended files, interfaces, risks and tests before implementation.

(iv) Implement — make the smallest coherent change.

(v) Verify — run every mandatory quality gate.

(vi) Repair — when a gate fails, identify root cause and correct implementation, configuration or test fixture without weakening the criterion.

(vii) Regress — rerun the failed gate and all affected upstream/downstream gates.

(viii) Evidence — store machine results and concise human-readable summaries.

(ix) Report — state PASS, FAILED or BLOCKED accurately; never infer acceptance from partial success.

(x) Handover — request the required human review or approval where the work package reaches a controlled decision boundary.

PART IV — READINESS AND ACCEPTANCE

Section 12 — Definition of Ready

A work package is Ready for Claude execution only when the following conditions are satisfied or explicitly marked not applicable.

(i) a unique work-package identifier and concise objective exist;

(ii) applicable source-of-truth documents and versions are known;

(iii) requirement IDs and quality attributes are identified;

(iv) package ownership and permitted change boundary are clear;

(v) known architecture questions are resolved or captured in an approved ADR;

(vi) acceptance criteria are objective and testable;

(vii) mandatory QA commands or the procedure for creating them are defined;

(viii) necessary test fixtures, reference backends, datasets or mocks are available;

(ix) security, privacy, accessibility, offline, reporting and localisation applicability has been considered; and

(x) no unresolved blocker makes implementation speculative.

(xi) where the work is design-significant, an approved design work package and handoff manifest exist; and

(xii) design assets, interaction notes, responsive variants and any open design decisions required for implementation are available and versioned.

Section 13 — Definition of Done

A work package is Done only when the implementation and its evidence satisfy all applicable criteria. “Code generated”, “build succeeds” or “Claude reports completion” is insufficient.

(i) all in-scope acceptance criteria have PASS evidence;

(ii) architecture/package-boundary checks pass;

(iii) strict type checking and compilation pass;

(iv) applicable unit/component/contract/integration tests pass;

(v) applicable accessibility, responsive and visual-regression checks pass;

(vi) applicable security and privacy checks pass;

(vii) applicable performance/reliability budgets pass;

(viii) new dependencies are approved and reflected in licence/SBOM evidence;

(ix) documentation, UI-lab examples or migration notes are updated where required;

(x) clean-room provenance and implementation rationale are retained for material design/code work;

(xi) no unresolved critical/high defect remains unless explicitly accepted through controlled exception; and

(xii) required human acceptance or review has been recorded.

(xiii) where a controlled handoff applies, implementation conformance evidence references the approved handoff version; and

(xiv) all approved Design Deviation Records are resolved or explicitly accepted before final work-package acceptance.

Section 14 — Acceptance Criteria Standard

Acceptance criteria shall be observable, binary where practical, linked to a requirement and written so a different engineer can independently verify the result. Criteria such as “looks good”, “works properly” or “professional UI” are not sufficient without measurable behaviour.

ID Pattern	Use	Example
AC-FUNC-*	Functional behaviour	Given a valid query, the ReportingGateway returns the defined typed result contract.
AC-ARCH-*	Architecture/boundaries	contracts compiles without Angular, Frappe, DOM or chart-vendor imports.
AC-SEC-*	Security/privacy	No browser bundle or runtime config contains privileged credentials.
AC-A11Y-*	Accessibility	Keyboard-only user can enter, operate and exit the dialog with focus restored.
AC-RWD-*	Responsive/adaptive UX	Critical workflow remains usable at 320 CSS px without page-level horizontal scrolling.
AC-PERF-*	Performance	Route/bundle or user timing stays within the work-package QAC budget.
AC-I18N-*	Localisation	Activated locale formats and long-text/RTL scenarios pass visual checks.
AC-OFF-*	Offline integrity	Replay suite produces zero duplicate business effects.
AC-PROV-*	Provenance/licence	New dependency has recorded rationale, licence and approval.

Section 15 — Quality Gate Matrix

Gate	Purpose	Typical Trigger	Release Blocking?
qa:format	Deterministic formatting	All source changes	Yes when configured
qa:lint	Code quality and forbidden patterns	All TS/HTML/CSS changes	Yes

Gate	Purpose	Typical Trigger	Release Blocking?
qa:architecture	Package direction, public APIs, cycles	Package/import changes	Yes
qa:types	Strict TS/template checking	All compile-time changes	Yes
qa:unit	Deterministic logic tests	Changed logic	Yes
qa:component	Component states/interactions	Reusable UI changes	Yes
qa:contracts	Public contract compatibility	Contracts/SDK/ reporting changes	Yes
qa:a11y	Automated accessibility checks	UI/component changes	Yes
qa:visual	Controlled visual regression	Design/UI changes	Yes for controlled surfaces
qa:security	Dependency/static/ security checks	All releases; security-sensitive changes	Yes
qa:performance	Budgets and representative scenarios	Shell/reporting/data-heavy changes	Yes when applicable
qa:integration	Adapter/backend contract behaviour	Frappe/reporting/ offline integration	Yes
qa:e2e	Golden workflows and recovery	Milestone/release candidates	Yes
qa:provenance	Dependency/SBOM/ provenance checks	Release candidates	Yes

PART V — QUALITY ASSURANCE DOMAINS

Section 16 — Architecture and Modularity Assurance

(i) Enforce dependency direction using lint/graph rules; architecture diagrams alone are not sufficient.

(ii) Fail CI on circular dependencies, prohibited upward imports and deep/internal package imports.

(iii) Keep @proseit/paep-contracts framework-neutral and free of Angular, Frappe, DOM and visualization-vendor dependencies.

(iv) Keep apps/* thin composition layers; reusable platform logic belongs in controlled packages.

(v) Require public package entry points and prohibit consumers from importing implementation-private paths.

(vi) Require an ADR when a work package introduces a new shared package, changes dependency direction or creates an independently deployable frontend boundary.

CLAUDE EXECUTION PROMPT — ARCHITECTURE CONFORMANCE REVIEW

Review the current change for PAEP architecture conformance only.
Check package ownership, dependency direction, public entry-point use, portability boundary, direct Frappe coupling and circular/deep imports.
Run the configured architecture gates. Do not modify architecture rules merely to make the change pass.
If a genuine architecture need exists, stop and draft an ADR proposal with options, consequences and migration impact.

Section 17 — Security and Privacy Assurance

(i) backend authorization remains authoritative for every consequential read/write;

(ii) no secrets or privileged credentials enter browser bundles or public runtime configuration;

(iii) framework-safe templating is preferred and untrusted HTML execution is prohibited unless a reviewed sanitisation path exists;

(iv) production builds follow the controlled AOT/CSP/Trusted-Types posture defined by PAEP deployment profiles;

(v) browser storage of sensitive data is minimised and lifecycle-controlled by tenant/user/session context;

(vi) file uploads/downloads rely on server authorization and validation; client constraints are usability aids only;

(vii) frontend logs and telemetry exclude raw sensitive payloads; and

(viii) negative tests prove that hidden or disabled UI is not the sole authorization control.

Section 18 — Accessibility, UX and Responsive Assurance

(i) automated accessibility checks are run for every reusable component and critical workflow;

(ii) manual keyboard/focus testing is mandatory for dialogs, drawers, menus, command interfaces, grids and other complex interactions;

(iii) critical content must reflow at the PAEP controlled narrow viewport target and remain usable under browser zoom;

(iv) enterprise tables shall adopt an explicit narrow-screen strategy rather than causing uncontrolled whole-page scrolling;

(v) focus, validation errors, status changes and destructive actions are exposed using accessible semantics and non-colour cues;

(vi) long translated strings and activated RTL layouts receive visual regression coverage; and

(vii) critical product workflows receive representative-user usability evidence before release acceptance.

(viii) design-significant screens have an approved handoff identifying desktop, tablet and narrow/mobile intent as applicable;

(ix) implementation comparison evaluates design intent, PAEP token/component use, state behaviour and responsive adaptation rather than relying on brittle pixel equality alone; and

(x) material visual or interaction deviations are recorded and reviewed rather than silently normalized by Claude Code.

Section 19 — Performance, Reliability and Observability

(i) performance targets are defined in QACs for representative devices, networks and data volumes;

(ii) route-level loading, deferred non-critical content, bounded DOM size, server paging/filtering and request cancellation are applied where justified;

(iii) retries distinguish safe/idempotent operations from consequential writes;

(iv) error boundaries and surfaced diagnostics retain release, route and correlation context without exposing sensitive payloads;

(v) reconnect/reload behaviour shall not duplicate business effects or silently discard local work; and

(vi) performance regressions outside approved budgets block the applicable release or require a controlled exception.

Section 20 — Reporting and Analytics Assurance

PAEP reporting shall support rich interactive analytics without turning the Angular browser into a database client or coupling PAEP public contracts to one visualization or BI vendor.

(i) all reporting requests use governed dataset, metric, dimension, filter, drill and export contracts;

(ii) browser code shall not contain SQL, analytical-store credentials or unrestricted backend query capability;

(iii) metric correctness is tested against controlled known-answer datasets;

(iv) tenant and row/field isolation receive negative tests;

(v) cross-filter, drill-down, drill-through and saved-view behaviour receive deterministic interaction tests;

(vi) large results use server-side aggregation/paging and asynchronous exports where appropriate;

(vii) charts provide accessible labelling and a tabular or equivalent alternative when needed;

(viii) internal visualization adapters may use approved libraries, but vendor configuration objects shall not appear in PAEP public reporting contracts; and

(ix) optional Power BI or other BI embedding remains behind an approved adapter and shall not become a mandatory PAEP runtime dependency.

Section 21 — Offline/PWA and Localisation Assurance

(i) offline-enabled workflows are explicitly enumerated; network absence does not bypass backend rules;

(ii) queued commands retain client IDs/idempotency metadata where duplicate effects are possible;

(iii) local data is partitioned by tenant/user/session context and cleared/expired according to policy;

(iv) service-worker upgrades are tested against unsynchronised local work;

(v) conflict handling exposes both attempted and authoritative state for reconciliation;

(vi) activated locales use locale-aware dates, numbers, currency and business timezone context; and

(vii) long-text and RTL checks are part of applicable visual regression suites.

Section 22 — Clean-Room, Dependencies and Provenance

(i) Claude prompts shall request capabilities and product-specific interaction design rather than replication of proprietary screens;

(ii) commercial-template source, theme files, assets and copied distinctive implementation artefacts are prohibited inputs;

(iii) every new dependency requires recorded package name, version, licence, rationale and approval status;

(iv) generated design/code is reviewed before adoption and material prompts or design rationale are retained where required;

(v) release-level SBOM evidence is generated in the approved machine-readable format; and

(vi) provenance, licence and unresolved-exception review forms part of the release gate.

PART VI — DEFECT, EVIDENCE AND CHANGE CONTROL

Section 23 — Fix-Until-Pass Defect Protocol

Mandatory criteria use a repair loop, not a negotiated partial-completion state. Claude shall not declare success because most tests pass. A work package remains IN PROGRESS or FAILED until all mandatory criteria pass or an authorised exception is recorded.

(i) reproduce the failing gate and retain the relevant failure output;

(ii) classify whether the defect lies in implementation, test fixture, configuration, environment or an ambiguous requirement;

(iii) fix the root cause without weakening the acceptance criterion;

(iv) rerun the specific failed gate;

(v) rerun all affected regression gates;

(vi) update implementation notes/evidence if the repair changes design or risk assumptions;

(vii) escalate when the failure exposes a controlled architecture conflict, unavailable dependency or requirement ambiguity; and

(viii) close the defect only when objective PASS evidence exists.

NO TEST LAUNDERING: Deleting a failing assertion, broadening a tolerance, ignoring an accessibility violation, disabling lint, adding unsafe casts, suppressing compiler diagnostics or excluding files merely to turn a red gate green is prohibited unless the changed criterion is separately reviewed and approved.

Section 24 — Evidence Repository and Retention

Recommended evidence layout

```
evidence/
└── <milestone-or-work-package>/
    ├── requirement-traceability.md
    ├── implementation-summary.md
    ├── qa/
    │   ├── lint.txt
    │   ├── architecture.txt
```

```
│   ├── types.txt
│   └── test-results/
├── accessibility/
├── visual/
├── performance/
├── security/
├── provenance/
└── acceptance-report.md
```

(i) evidence must identify the source commit/release under test;

(ii) machine-readable reports are preferred where tooling supports them;

(iii) human-readable summaries shall identify failures, waivers, outstanding risks and reviewer decisions;

(iv) visual regression baselines shall be controlled and intentional changes approved rather than bulk-updated automatically;

(v) security/accessibility reports containing sensitive details shall use appropriate restricted access; and

(vi) release evidence shall remain linked to the controlled release manifest and compatibility matrix.

Section 25 — Requirements Traceability

Trace Field	Required Content
Requirement ID	Unique controlled requirement identifier
Source	PAEP section / baseline / ADR / product requirement
Work Package	Implementing WP or milestone
Implementation Artifact	Package/file/component/API
Acceptance Criteria	One or more AC IDs
Verification	Test/gate/report identifier
Status	NOT STARTED / IN PROGRESS / BLOCKED / FAILED / PASS / ACCEPTED
Evidence Location	Repository path or controlled evidence record

Section 26 — ADR and Architecture Exception Control

(i) Claude may identify the need for an ADR but shall not treat a draft ADR as approval.

(ii) An ADR shall state context, decision, alternatives considered, consequences, affected packages/contracts, migration impact and rollback/exit implications.

(iii) An architecture exception shall identify the exact control being waived, scope, owner, justification, risk, compensating control and expiry/sunset condition.

(iv) Approved exceptions shall be visible to quality gates where necessary so future engineers do not mistake an intentional deviation for an undocumented defect.

(v) Permanent architecture changes require incorporation into the appropriate controlled specification, not indefinite reliance on exception records.

Section 27 — Defect Classification and Escalation

Severity	Typical Meaning	Default Treatment
Critical	Security breach path, tenant isolation failure, data loss, duplicate consequential transaction, architecture control fundamentally bypassed	Stop release immediately; escalate; fix before acceptance
High	Major workflow failure, accessibility blocker in critical path, contract incompatibility, material performance breach	Release blocking unless formal exception
Medium	Non-critical functional or UX defect with workaround	Fix before milestone close unless accepted into controlled backlog
Low	Minor cosmetic/documentation issue without functional risk	May be scheduled with Product Owner approval

PART VII — PAEP EXECUTION ROADMAP

Section 28 — W0-W12 Controlled Work Breakdown

WBS	Claude Delivery Focus	Mandatory Gate Theme
W0	Traceability, ADR/dependency/provenance registers, QACs, CLAUDE.md	Governance readiness
W1 / IB0	Workspace, package projects, CI, boundary rules, UI lab/reference app skeletons	Build, types, architecture, CI reproducibility
W2 / IB1	Contracts, reporting contracts, runtime schema/validator, fail-safe bootstrap	Framework-neutral contracts, config validity, secret checks
W3 / IB2	Tokens, responsive foundations, core components, UI lab documentation	Component, a11y, responsive, visual regression
W4	Enterprise shell, navigation, context, error/unavailable/connectivity states	Shell integration, keyboard, routes, config/error states
W5	Frappe adapter for auth/workflow/files/realtime/errors/concurrency	Contract/integration/security-negative tests
W6	Governed analytics/reporting UI and adapters	Metric correctness, isolation, drill/filter/export, performance
W7	Offline/PWA/i18n/low-bandwidth resilience	Fault injection, duplicate prevention, cache isolation, locale/RTL
W8	Reusable domain kits	Cross-product reuse + no embedded product-specific business rules
W9	Golden vertical slice	End-to-end workflow + recovery + observability

WBS	Claude Delivery Focus	Mandatory Gate Theme
W10	System qualification	Security, a11y, performance, browser/device, portability
W11	Release packaging	SBOM, provenance, compatibility, migration, rollback
W12	Product adoption/evolution	Migration/UAT/decommission and ongoing compatibility

Section 29 — IB0-IB2 Execution Protocol

29.1 IB0 — Workspace and Architecture Governance Baseline

(i) bootstrap the pinned Angular workspace and configured orchestration model;

(ii) create apps/reference-app and apps/ui-lab plus the approved package skeleton;

(iii) configure strict TypeScript/template checks, formatting and linting;

(iv) implement architecture tags, dependency-direction rules and cycle/deep-import detection;

(v) configure reproducible CI from the committed lockfile; and

(vi) capture a baseline showing clean install, build, type check, lint, architecture checks and starter-app launch all PASS.

CLAUDE EXECUTION PROMPT — IB0 EXECUTOR

Implement only PAEP IB0.
Create the governed workspace and architecture-boundary enforcement before feature development.
Do not add product screens or Frappe integration.
Mandatory exit: clean install, production build, strict types, lint, architecture boundaries, cycle detection, public entry points, reference-app startup and ui-lab startup all PASS.
If any gate fails, remain IN PROGRESS and fix until PASS.

29.2 IB1 — Product-Neutral Contracts and Runtime Configuration

(i) implement @proseit/paep-contracts as plain TypeScript with core, config, identity, domain, reporting and offline contracts;

(ii) ensure contracts contain no Angular, Frappe, DOM, SQL or visualization-vendor dependency;

(iii) implement the versioned runtime configuration schema and generated/controlled validator;

(iv) implement immutable runtime configuration access and fail-safe startup for invalid/incompatible configuration;

(v) add positive and negative configuration tests, including secret-detection controls; and

(vi) capture compatibility evidence showing the contract layer remains portable while Frappe remains the strategic backend adapter.

CLAUDE EXECUTION PROMPT — IB1 EXECUTOR

Implement only PAEP IB1 on the accepted IB0 workspace.
Treat @proseit/paep-contracts as the portability boundary.
Do not connect UI components directly to Frappe and do not introduce vendor chart objects into reporting contracts.
Mandatory exit: contracts compile independently; forbidden imports are zero; runtime config valid/invalid cases are tested; fail-safe startup works; secret checks PASS.

29.3 IB2 — Foundation, Core Components and UI Laboratory

(i) implement semantic design tokens for typography, colour, spacing, size, radius, elevation, motion, breakpoints, density, focus and state;

(ii) implement the first bounded component set across actions, forms, data, feedback, navigation, overlays and structural layout;

(iii) document component variants and states in ui-lab independently of product-specific screens;

(iv) add keyboard/focus, automated accessibility, responsive/reflow, long-text, RTL-readiness and visual-regression tests as applicable;

(v) reject raw uncontrolled design values in reusable components where semantic tokens exist; and

(vi) complete IB2 only when every component in scope has green tests, documented states and retained evidence.

(vii) establish the PAEP design-system source that Claude Design may consume after the initial tokens/components are accepted;

(viii) classify IB2 and later UI work as Design Significant, Design Extension or Direct-Code Maintenance using PAEP-DCH-01; and

(ix) require controlled handoff evidence before Claude Code implements design-significant enterprise screens, dashboards or complex workflows.

CLAUDE EXECUTION PROMPT — IB2 EXECUTOR
Implement only the approved IB2 component slice. Use semantic PAEP tokens, not ad-hoc product styling. Every reusable component must include required states, keyboard/focus behaviour, accessibility semantics, responsive behaviour and ui-lab documentation. Run component, a11y, responsive and visual-regression gates. Do not bulk-update visual baselines to hide unintended changes. Remain IN PROGRESS until all mandatory component evidence is PASS.

Section 30 — Post-IB2 Execution Controls

After IB2, the same protocol continues through shell, Frappe integration, analytics/reporting, offline/i18n, domain kits and the golden vertical slice. Later milestones add integration complexity but do not relax the underlying requirements for traceability, bounded prompts, package discipline, executable gates and retained evidence.

(i) W4 shall assemble the enterprise shell from accepted foundation/components rather than duplicating local primitives.

(ii) W5 shall implement Frappe only behind PAEP contracts and prove negative authorization and concurrency/idempotency scenarios.

(iii) W6 shall provide rich reporting through governed reporting contracts and server-authoritative datasets/metrics.

(iv) W7 shall fault-inject offline and connectivity failure scenarios rather than validating only happy-path caching.

(v) W8 shall require genuine reuse evidence before moving an interaction into a shared domain kit.

(vi) W9 shall prove one golden end-to-end workflow before claiming platform readiness.

(vii) W10-W11 shall qualify the release across security, accessibility, performance, compatibility, SBOM/provenance and rollback readiness.

(viii) W4-W9 design-significant user experiences shall follow Design Brief → Claude Design → Design Review → Approved Handoff → Claude Code → QA → Acceptance.

(ix) Claude Design handoff bundles may be used as transport artifacts, but the PAEP handoff manifest and acceptance evidence remain the controlled record.

PART VIII — CONTROLLED TEMPLATES AND APPENDICES

Section 31 — Work-Package Control Template

Control Field	Required Entry
Work Package ID	<WP-ID>
Milestone / WBS	<IB0 / W4 / etc.>
Objective	One bounded engineering outcome
Source-of-truth	Controlled sections and ADRs
Requirement IDs	<REQ-...>
Permitted change boundary	Packages/directories/files
Out of scope	Explicit exclusions
Architecture constraints	Only material controls
Acceptance criteria	AC IDs and measurable pass conditions
Mandatory gates	Exact commands / manual checks
Evidence path	Controlled repository location
Required reviewers	Architecture / QA / security / Product Owner as applicable
Status	NOT STARTED / IN PROGRESS / BLOCKED / FAILED / PASS / ACCEPTED

Section 32 — Acceptance Report Template

Acceptance Field	Required Content
Work package	ID, title, commit/release
Implemented scope	Concise summary of changed behaviour/artifacts
Requirement traceability	Requirement IDs → implementation → tests
Automated gates	Command, result, report/evidence location
Manual checks	Reviewer, date, scope and result
Defects	Open/closed defects by severity
Exceptions	Approved exception IDs or “None”
Provenance	Dependency/design/code-assistance evidence status

Acceptance Field	Required Content
Residual risk	Known limitations and planned follow-up
Technical result	PASS / FAILED / BLOCKED
Acceptance decision	ACCEPTED / REJECTED / CONDITIONAL by authorised human role

Section 33 — Claude Prompt Library

33.1 Defect repair prompt

CLAUDE EXECUTION PROMPT — DEFECT REPAIR

A mandatory PAEP quality gate has failed.
Reproduce the failure, identify the root cause and classify it as implementation, fixture, configuration, environment or requirement ambiguity.
Fix the cause without deleting/weakening the criterion or suppressing the diagnostic.
Re-run the failed gate and affected regression gates.
Report the evidence path and leave status FAILED/BLOCKED if PASS cannot be achieved without an architecture or requirement decision.

33.2 Review-only prompt

CLAUDE EXECUTION PROMPT — INDEPENDENT QA REVIEW

Do not modify code initially.
Review this work package against its acceptance criteria and applicable PAEP controls.
Run the specified quality gates and report each criterion as PASS, FAIL, BLOCKED or NOT APPLICABLE with evidence.
If a defect exists, describe the smallest corrective action. Do not waive or reinterpret criteria to produce a pass.

33.3 ADR proposal prompt

CLAUDE EXECUTION PROMPT — ARCHITECTURE DECISION PROPOSAL

A current work package appears to require a PAEP architecture decision.
Do not implement the architecture change.
Draft an ADR proposal containing context, decision question, at least two viable options, consequences, affected packages/contracts, security/accessibility/portability/reporting implications, migration impact

and recommended option.
State which current gate or requirement is blocked pending approval.

Section 34 — Release Readiness Decision

(i) all milestone/work-package acceptance reports are present and required statuses are ACCEPTED;

(ii) compatibility matrix covers supported PAEP, backend contract and deployment profiles;

(iii) critical/high defects are closed or covered by authorised release exceptions;

(iv) security, accessibility, performance and portability qualification gates pass;

(v) release SBOM, licence register and clean-room provenance attestation are complete;

(vi) migration and rollback procedures are documented and testable;

(vii) production runtime configuration schema/version is compatible with the release;

(viii) reference application/golden workflow passes on representative profiles; and

(ix) Solutions Architect and Main Product Owner record the controlled release decision.

Appendix A — Baseline CLAUDE.md

```
# PAEP CLAUDE ENGINEERING CONTRACT

You are implementing the controlled ProSEIT Angular Experience Platform (PAEP).

SOURCE-OF-TRUTH PRIORITY
1. PAEP v1.1 Controlled Platform Specification
2. PAEP Implementation Architecture & Repository Baseline
3. Approved Architecture Decision Records (ADRs)
4. Current Work-Package Specification
5. Acceptance Criteria and Executable Quality Gates

YOU SHALL NOT
- change controlled architecture decisions without approval;
```

```
- introduce direct UI-to-Frappe persistence coupling;
- introduce prohibited package dependencies, cycles or deep imports;
- embed secrets in frontend bundles or runtime configuration;
- expose SQL or analytical-store credentials to browser code;
- make visualization-vendor objects part of PAEP public reporting contracts;
- weaken, delete, skip or suppress tests to obtain PASS;
- copy proprietary template source/assets/distinctive implementation;
- mark a work package COMPLETE while a mandatory gate is failing or unexecuted.

FOR EACH WORK PACKAGE
1. Read controlled requirements and relevant ADRs.
2. Identify requirement and acceptance IDs.
3. Produce a bounded implementation plan.
4. Implement the smallest coherent change.
5. Run every mandatory gate.
6. On failure, diagnose and fix the root cause.
7. Re-run the failed gate and affected regression set.
8. Retain evidence and implementation notes.
9. Report status accurately.
10. Request required human approval.

MANDATORY STATUS RULE
A failed mandatory criterion means the work package remains IN PROGRESS or FAILED.
Only an approved controlled exception may waive a mandatory gate.
```

Appendix B — QA Command Naming Standard

Recommended script namespace; concrete commands are implemented by repository tooling

```
npm run qa:format
npm run qa:lint
npm run qa:architecture
npm run qa:types
npm run qa:unit
npm run qa:component
npm run qa:contracts
npm run qa:a11y
npm run qa:visual
npm run qa:security
npm run qa:performance
npm run qa:integration
```

```
npm run qa:e2e
npm run qa:provenance

npm run qa:ib0
npm run qa:ib1
npm run qa:ib2
npm run qa:shell
npm run qa:frappe
npm run qa:analytics
npm run qa:offline
npm run qa:release
```

PRINCIPLE: Exit code 0 denotes technical PASS for an executable gate. A non-zero exit denotes FAIL. Technical PASS is necessary but does not replace required human acceptance.

Appendix C — Status Model

Status	Meaning	Who/What Sets It
NOT STARTED	Work package has not entered execution	Project/work-package control
IN PROGRESS	Implementation or verification is active; no completion claim	Engineer/Claude workflow
BLOCKED	External decision, environment, dependency or authority prevents progress	Engineer with blocker evidence
FAILED	One or more mandatory criteria failed	Quality gate / reviewer
PASS	All mandatory technical criteria in scope passed	Executable gates + required QA evidence
ACCEPTED	Competent human authority accepts the work package/release	Product Owner / Solutions Architect / delegated reviewer

Appendix D — Controlled References

(i) ProSEIT Angular Experience Platform (PAEP) v1.1, PROSEIT-STD-PMRA-AF-006.

(ii) PAEP Implementation Architecture & Repository Baseline v1.0, PAEP-IB-01.

(iii) PAEP Implementation Roadmap: Milestones IB0-IB2 v1.0, PAEP-IR-01.

(iv) PMRA-AF Version 2.0 and its controlled standards register where inherited by PAEP.

(v) Approved PAEP ADRs, QACs, compatibility matrices, release manifests, dependency registers and controlled work-package specifications.

(vi) PAEP Design-to-Code Handoff Standard v1.0, PAEP-DCH-01, for design-significant work and controlled design implementation handoff.

REFERENCE CONTROL: Where this Protocol and a higher controlled PAEP/PMRA instrument conflict, the higher instrument prevails. The inconsistency shall be reported and reconciled through controlled document revision rather than silently interpreted away.

Protocol Approval and Activation Record

Complete this record only after the Protocol has been reviewed against PAEP v1.1, PAEP-IB-01 and the approved implementation roadmap. Activation authorises use of this execution protocol; it does not delegate architecture or release authority to Claude.

CONTROL FIELD	RECORDED VALUE
Protocol ID	PROSEIT-STD-PMRA-AF-006-EQP01 / PAEP-EQP-01
Parent architecture	PAEP v1.1 - PROSEIT-STD-PMRA-AF-006
Implementation baseline	PAEP-IB-01 v1.0
Architecture review	[INSERT REFERENCE AND DATE]
Quality/security review	[INSERT REFERENCE AND DATE]
Approved repository CLAUDE.md version	[INSERT]
Effective date	[INSERT]
Next review date	[INSERT]

Authorised Signatures

Solutions Architect / Main Product Owner — signature and date:

_____

Engineering / Frontend Platform Lead — signature and date:

_____

Security / Quality reviewer — name, capacity, signature and date:

_____